



Санкт-Петербургский государственный университет
Кафедра системного программирования

Сравнение производительности алгоритмов на графах из SPLA/GraphBLAS+LAGraph

Гавриленко Михаил, группа 23.Б15-мм

Санкт-Петербург
2023

Постановка задачи

Целью является исследование алгоритмов BFS, SSSP, TC, PR из библиотек SPLA и GraphBLAS/LAGraph

Задачи:

- Сформулировать набор гипотез о производительности алгоритмов на основе базовых представлений о наивном алгоритме и особенностях платформ
- Произвести замеры производительности реализаций из выбранных библиотек
- Исследовать результаты и выявить особенности, отразившиеся на производительности
- Сделать выводы о ранее сформулированных гипотезах

Использованные метрики:

- Время выполнения (мс)
- Степень прироста производительности SPLA относительно LAGraph
- GTEPS(Giga Traversed Edges Per Second)

Конфигурация системы:

- M2 Max (arm64), 38-гпу ядер 1000МГц, 12-спу ядер (8+4), 32ГБ ОЗУ
- OpenCL 1.2
- clang 17.0.0

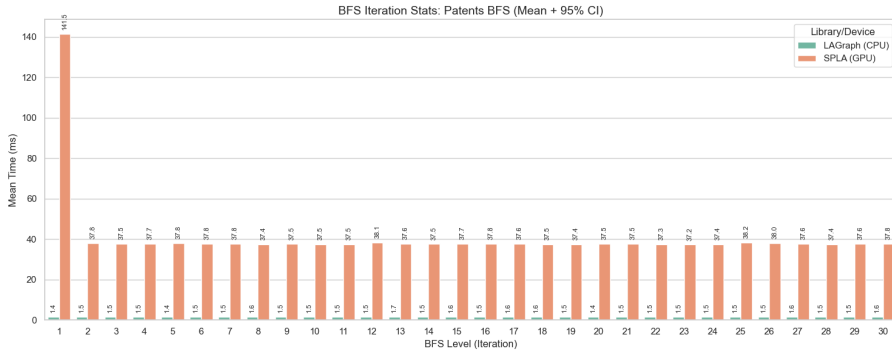
BFS: (первичные тезисы)

- Исполнение алгоритма эффективнее на GPU
- GPU лучше справляется с симметричными датасетами, в отличие от CPU
- Преобразование в неориентированный граф увеличит вычислительную нагрузку, но улучшит показатель GTEPS за счет более эффективной работы Pull-фазы (bottom-up BFS)
- Первая итерация на GPU будет значительно медленнее последующих из-за JIT-компиляции ядер OpenCL

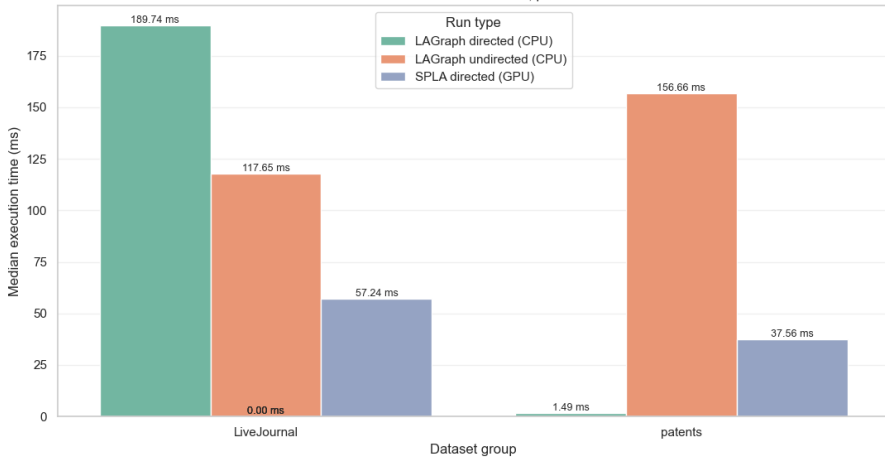
BFS: (детали алгоритмов)

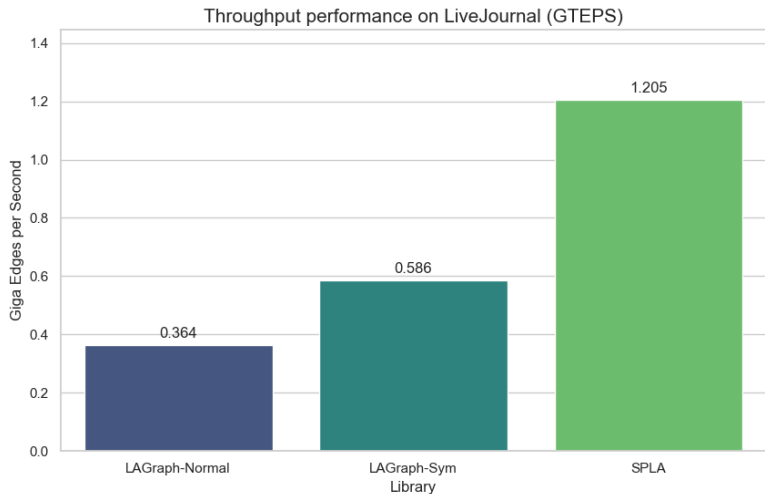
Оба алгоритма являются "pull-push", однако некоторые детали различаются:

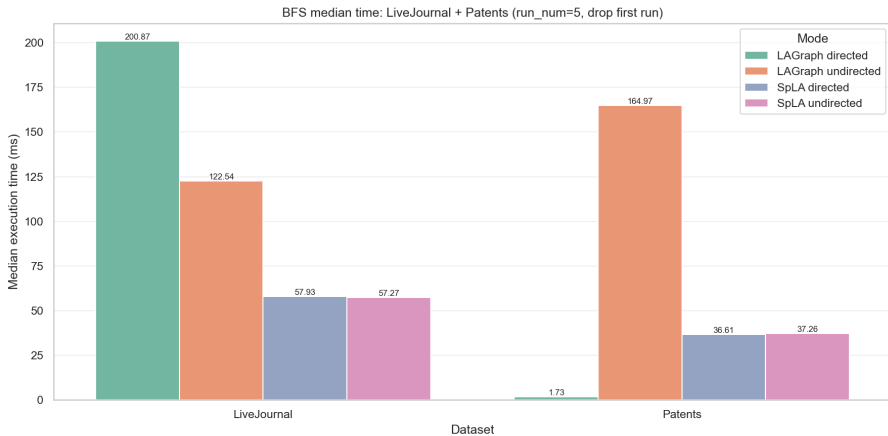
- SPLA выбирает push/pull по плотности текущей фронтальной структуры
- LAGraph опирается на кэшированные свойства, в ином случае переключается на push-only
- В SPLA есть явный early-exit



Median BFS time: LiveJournal first, patents second







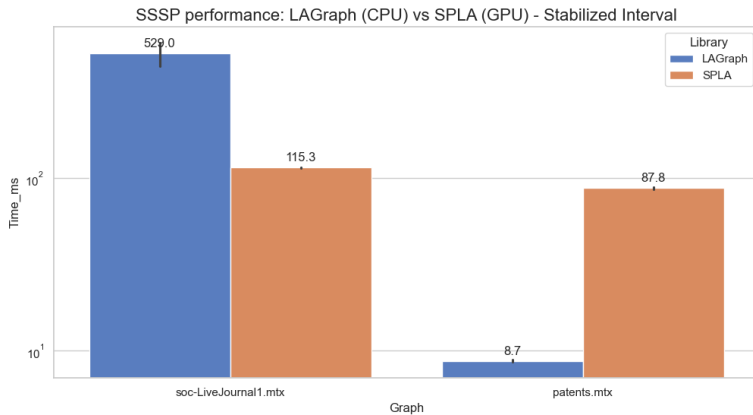
- Push - обходит граф по направлению ребер.
- Pull - обходит граф против направления ребер.
- Алгоритм пытается проехать по улице с односторонним движением навстречу потоку.
- Он найдет вершины, из которых можно прийти во фронт, а не те, в которые можно попасть из фронта.
- Чтобы алгоритм Direction-Optimizing BFS (переключение Push/Pull) корректно работал на направленных графах, для шага Pull необходимо использовать транспонированную матрицу.

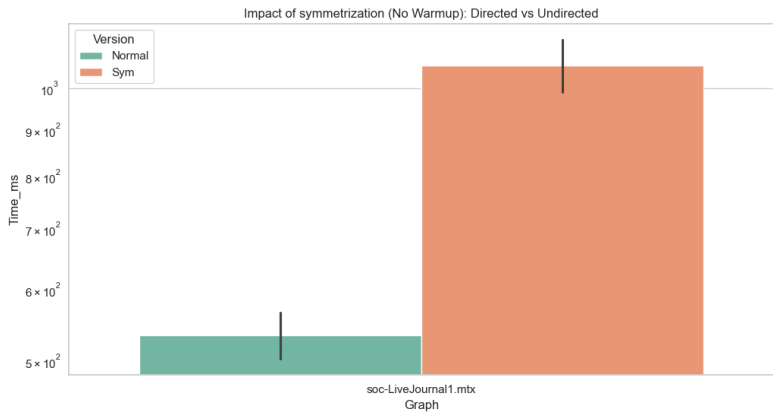
```
if (push || (push_pull && is_push_better)) \{  
    exec_vxm_masked(frontier_new, v, frontier_prev, A, BAND_INT, BOR_INT, EQZERO_INT  
\} else \{  
    exec_mxv_masked(frontier_new, v, A, frontier_prev, BAND_INT, BOR_INT, EQZERO_INT  
\}
```

Выводы:

- Выполнение большого кол-ва итераций с маленьким фронтиром - отрицательно действует на производительность GPU
- Время обхода направленного графа зависит от его свойств - "длины" путей и "ширины" фронтиров. Высокий коэффициент кластеризации и малый диаметр соцсети позволяют быстро расширять фронт и эффективно использовать параллелизм
- SPLA эффективнее на больших графах с большой компонентой связности. На малых графах или при высокой фрагментации LAGraph предпочтительнее из-за низкой задержки CPU
- SPLA с оптимизацией push-pull работает только на ненаправленных графах.

- Исполнение алгоритма эффективнее на GPU
- GPU лучше справляется с симметричными датасетами, в отличии от CPU GPU (SPLA) покажет значительное преимущество на графах с плотной структурой и широким фронтиром
- SSSP в SPLA не работает с ориентированными графами



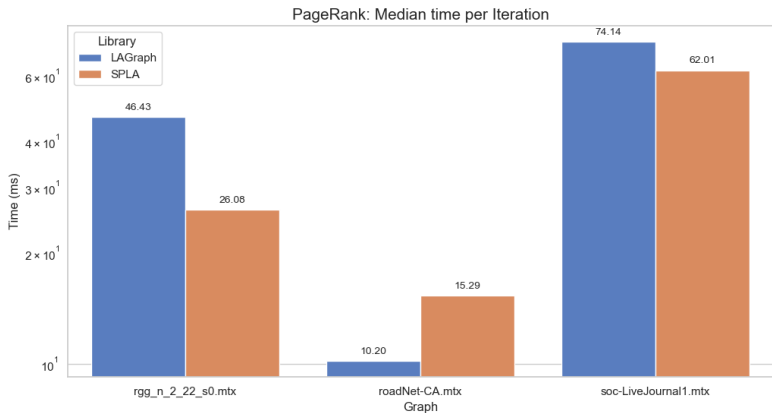


```
if (push || (push_pull && is_push_better)) \{
    exec_vxm_masked(frontier, dummy_mask, feedback, A, PLUS_FLOAT, MIN_FLOAT, ALWAYS
\} else \{
    exec_mxv_masked(frontier, dummy_mask, A, feedback, PLUS_FLOAT, MIN_FLOAT, ALWAYS
\}
```

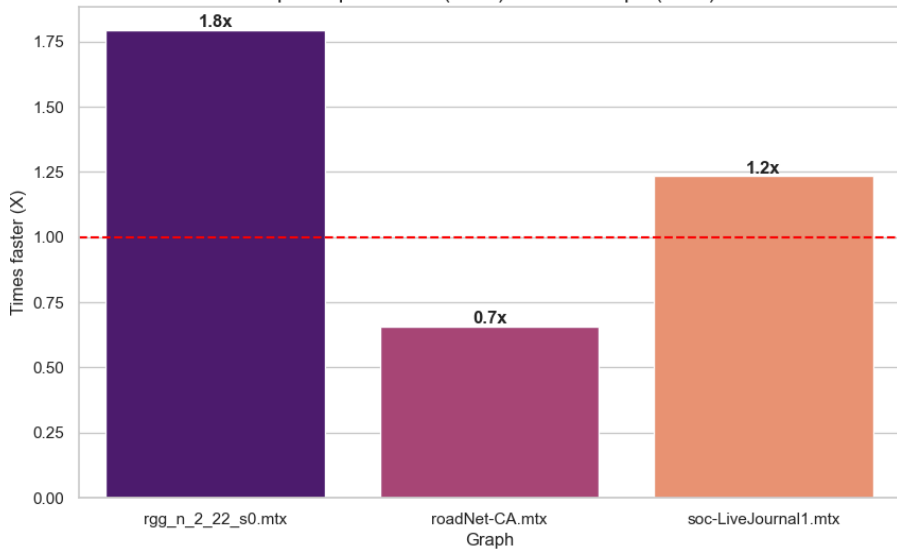

- В графе патентов (где 3.7 млн узлов) вершина с наибольшим кол-вом исходящих ребер связана только с 776 другими вершинами. У LiveJournal вершина имеет в 20 раз больше выходящих ребер
- На LiveJournal GPU работает в 4.7 раза быстрее CPU. Это "родная" среда для массивного параллелизма SPLA
- Ситуация с поддержкой ориентированных графов - точно такая же, как и в BFS. (алгоритм использует hybrid push/pull execution)

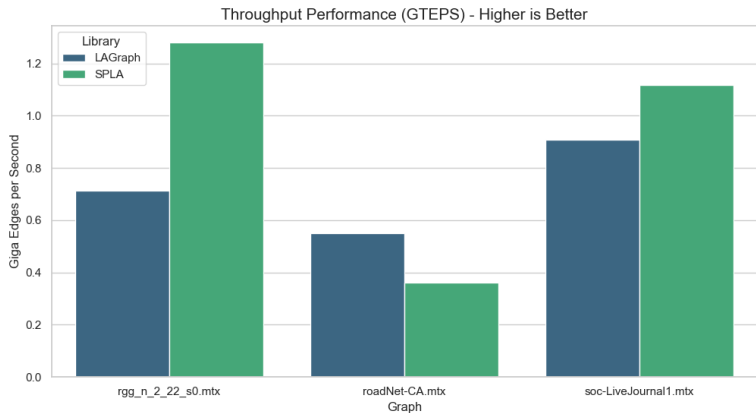
Общий вывод: Для алгоритма SSSP выбор вычислительного устройства критически зависит от топологии.

- Эффективность GPU-ускорения прямо пропорциональна регулярности структуры графа
- Низкая средняя степень узла делает использование GPU нецелесообразным
- Неравномерность распределения в безмасштабных сетях снижает потенциал GPU по сравнению с регулярными структурами



Speedup of SPLA (GPU) over LAGraph (CPU)

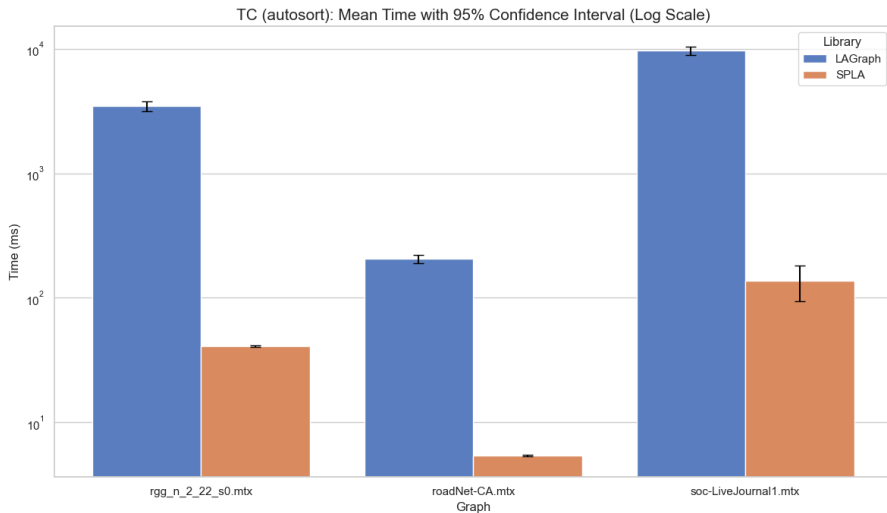




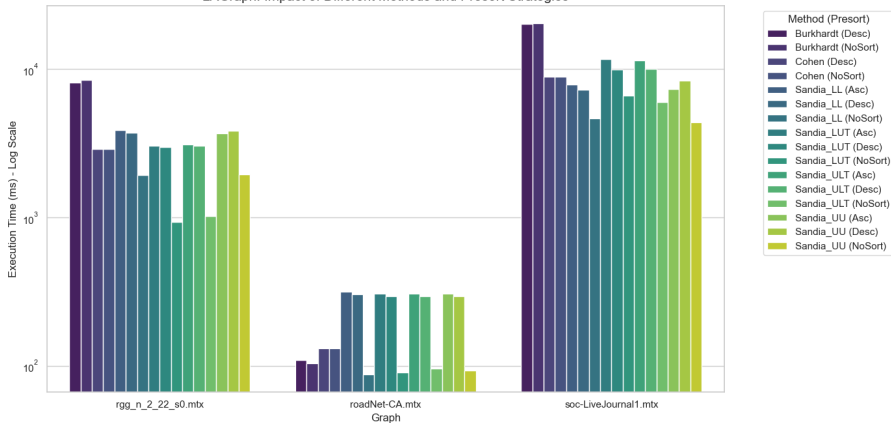
- Степени узлов распределены равномерно, что минимизирует простой ядер GPU
- Дорожные графы эффективнее на CPU. Накладные расходы на запуск ядер и обращение к памяти GPU не окупаются малым объемом вычислений на одно ребро.

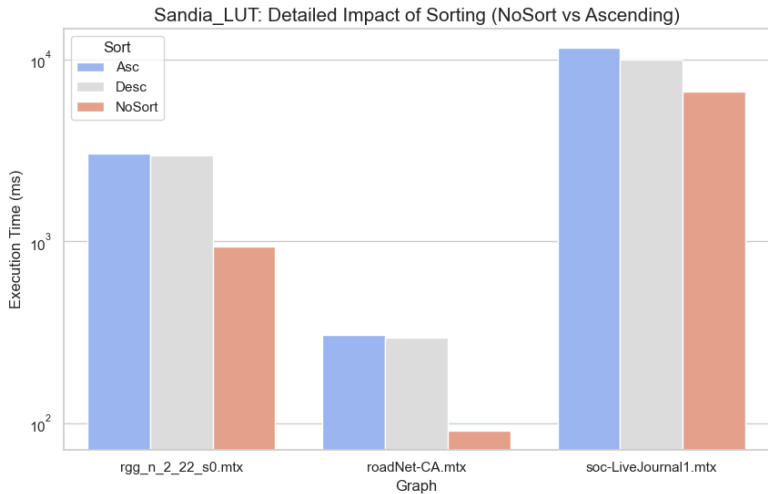
Общий вывод: SPLA эффективен на "тяжелых" графах с большим количеством ребер, но его преимущество нивелируется при сильной разреженности (дороги) или экстремальной неравномерности (социальные сети).

- Архитектура GPU должна показывать многократное преимущество в данной задаче
- На разреженных графах с низкой средней степенью/ (road) эффективность GPU будет ниже, чем на плотных или регулярных из-за накладных расходов
- Для алгоритмов семейства Sandia на CPU предварительная сортировка узлов по степени критически важна для минимизации количества вычислений



LAGraph: Impact of Different Methods and Presort Strategies





Основные операции в TC Sandia

- Взять список сетей U
- Взять список сетей V
- Найти пересечение

Сортировка является ключевой оптимизацией в случае семейства алгоритмов Sandia. Сортировка уменьшает стоимость одного пересечения увеличивая cache locality, меньше случайного доступа. Условия включения сортировки при выбранном режиме AutoSort.

- $n > 2000$
- $\text{density} > 10$
- $\text{mean} > 3 * \text{median}$ (определяет насколько граф powel-law)

Name	Mean	Median	Ratio	Density
rgg_n_2_22_s0	14,4	14	1	14,47
soc-LiveJournal1	14,2	4	3,5	14,12
roadNet-CA	2,79	3	0,9	2,8

Таблица: LAGraph TC, Graphs details

- На основе предыдущих графиков можем сделать вывод, что ни на одном графе (даже на soc-LiveJournal, который явно подходит под AutoSort) мы не получили прироста.
- Наоборот, при выборе NoSort - производительность оказалась выше, чем при принудительно Asc и Desc.
- Причина падения производительности может заключаться в затратах на предварительную сортировку графа.

```

//-----
// sort the input matrix, if requested
//-----

if (presort != LAGr_TriangleCount_NoSort)
\{
    // P = permutation that sorts the rows by their degree
    LG_TRY (LAGr_SortByDegree (&P, G, true,
        presort == LAGr_TriangleCount_Ascending, msg)) ;

    // T = A (P,P) and typecast to boolean
    GRB_TRY (GrB_Matrix_new (&T, GrB_BOOL, n, n)) ;
    GRB_TRY (GrB_extract (T, NULL, NULL, A, (GrB_Index *) P, n,
        (GrB_Index *) P, n, NULL)) ;
    A = T ;

    // free workspace
    LG_TRY (LAGraph_Free ((void **) &P, NULL)) ;
\}

```

Symbol	NoSort	Asc	Desc
mxm	25,5% (5,71s)	22,9% (5,72s)	23,1% (5,76s)
Matrix_extract	None	9,4% (2,36s)	9,8% (2,44s)
Matrix_reduce	0,3% (0,06s)	None	None
SortByDegree	None	1,4% (0,36s)	1,5% (0,36s)

Таблица: LAGraph TC, sort variations

После сортировки:

- маленькие узлы пересекаются друг с другом (дёшево)
- хабы концентрируются и реже пересекаются с другими хабами

- Ускорение на GPU достигает 84.5x. Задача Triangle Counting, основанная на пересечении множеств, идеально ложится на архитектуру SPLA
- Предположение о том, что Ascending Sort всегда ускоряет ТС на CPU, опровергнуто. Несмотря на схождение требований эвристик с характеристиками графа soc-LiveJournal1 - не получили прироста.
- Самой быстрой стратегией для ТС на CPU является использование наиболее эффективного алгоритма (Sandia LUT или LL) на исходном порядке узлов (NoSort), чтобы избежать дорогостоящих операций перестановки матриц
- Высокий GTEPS на дорогах говорит о том, что маскирование в SPLA работает исключительно эффективно даже при малом количестве треугольников